

AGRIAI 2.0 | FORENSIC SOFTWARE EXAMINATION

System Architecture and Construction

How the framework, frontend, backend and deployment units were assembled

Volume 01

Prepared and authored solely by Thaddeus Nii Teiko Tagoe

Founder, designer and sole creator of AgriAI

Computer Science, University of Ghana

Repository examined: ThaddeusHackz/agriai-v2

Evidence commit: 991457272b7fec548e02c645c5593b930af5ea29

Examination date: 18 August 2026

Classification: Internal technical record

Technical basis

This document is grounded in direct source review, route enumeration, dependency audit, secret pattern scanning, strict TypeScript checking, ESLint, production compilation, PostgreSQL integration tests and local HTTP behavior tests. Live third party AI accounts were not available in the examination environment, so provider success is assessed from code paths and contracts rather than a claim of live production availability.

1. Construction model

The codebase uses the Next.js App Router as both web framework and application server. There is no Express entry point. A route.ts file under app/api becomes an HTTP endpoint. A page.tsx file becomes a page. Client components use relative URLs, so the same origin receives UI and API traffic. The design keeps browser code independent from vendor hostnames and keeps secret keys in the Node.js runtime.

2. Technology bill of materials

Layer	Technology	Repository role
Framework	Next.js 16.2.10	Routing, rendering, route handlers, build and production server
View	React 19.2.4	Public product, streaming chat and admin single-page shell
Language	TypeScript 5	Strict types across UI, route handlers and libraries
Styling	Tailwind CSS 4 and PostCSS	Theme, responsive layout and utility classes
Primary AI SDK	@google/genai 2.16	Gemini text, streaming, vision, audio transcription and image attempts
Fallback AI	Cloudflare REST	Llama chat and Flux image generation
Persistence	Node fs and pg	Atomic local JSON plus PostgreSQL JSONB mirror
Authentication	bcryptjs and random opaque cookies	Password hashing and server-stored sessions
Search	Tavily, Wikipedia and DuckDuckGo	Grounding context and source metadata
Weather	OpenWeatherMap and Open-Meteo	Current conditions and five-day forecasts
Voice	Browser SpeechRecognition, Gemini, ElevenLabs	Speech input and audio output
UI libraries	Framer Motion, Lucide, Sonner, react-markdown	Motion, icons, notices and answer rendering

3. Repository anatomy

Path	Engineering responsibility
app/page.tsx	Composes the public product sections and sends visit tracking
app/layout.tsx	Global metadata, fonts, providers and application shell
app/admin	Login page and authenticated dashboard host
app/api	27 route files and 40 method handlers
components/Chat.tsx	SSE client, modes, languages, search toggle, microphone, TTS and feedback
components/DiseaseDetector.tsx	Crop photo preview, upload and disease result presentation
components/CropStudio.tsx	Agricultural image prompt and data URL download
components/admin	Dashboard, content, AI, price, knowledge, message, user and settings tabs
lib/ai.ts	Gemini adapters, model waterfall, local answers, disease parser and image generation
lib/cloudflare.ts	Workers AI HTTP adapter
lib/search.ts	Tavily and keyless search adapter
lib/db.ts	Schema seed, in-memory cache, JSON persistence and analytics mutation
lib/pg-store.ts	PostgreSQL connection, table creation and document upsert
lib/auth.ts	bcrypt, rate limit, sessions, cookies and role helpers
instrumentation.ts	Server boot PostgreSQL initialization and conditional hydration
render.yaml	Render web service, PostgreSQL and secret declarations
Dockerfile	Node 22 Alpine build and runtime image

4. Public page assembly

app/page.tsx renders AnnouncementBar, Nav, Hero, Chat, Features, MarketPrices, WeatherSection, DiseaseDetector, HowItWorks, CropStudio, Testimonials, Founder, FAQ, Newsletter and Footer. SiteProvider in lib/site-context.tsx retrieves /api/config, merges public settings and writes CSS custom properties. Each optional section receives a show flag from settings.showSections. This permits display changes without rebuilding the application.

```
app/layout.tsx
-> SiteProvider
  -> GET /api/config
  -> applyTheme(primaryColor, deepColor, accentColor)
-> app/page.tsx
  -> public section components
  -> relative fetch calls to /api/*
```

5. Server boundary

All API route files explicitly use the Node.js runtime. AI and storage routes set maxDuration values of 30 or 60 seconds. Browser requests never require CORS to call AgriAI because they use the same origin. Route handlers import server-only modules such as fs, crypto, pg and provider secret readers. The Next.js build marks every API route as dynamic and the public and admin pages as statically generated shells.

6. UI state and data flow

- * Chat state is local React state. Each browser stores an agriai_session_id in localStorage.
- * Admin state is tab-local. AdminShell calls /api/admin/me and redirects to login if the cookie is invalid.
- * Site settings load after the static shell mounts. Defaults in site-context prevent a blank first render.
- * Disease images remain in browser memory and are posted as base64 JSON. They are not written to a server gallery.
- * Generated images return as data URLs and the browser creates a temporary download link.
- * Market and weather components use no-store fetches to avoid stale browser content.

7. Route construction statistics

Category	Route files	Handlers	Examples
AI and media	6	6	chat, vision, generate, transcribe, tts, search
Public data and forms	10	13	config, weather, prices, history, contact, subscribe
Admin and auth	11	21	login, settings, users, knowledge, analytics
Total	27	40	All handlers use the App Router route.ts convention

8. Build and deployment shape

render.yaml provisions a Node web service and a PostgreSQL database. npm ci --include=dev and npm run build create the production output. npm start launches next start. DATABASE_URL comes from the managed database. Provider keys and admin seed values are declared sync:false. The Dockerfile offers a three-stage Node 22 Alpine build, copies .next, public, node_modules, package.json and next.config.ts, and starts the same Next.js server.

Deployment caveat

The Render healthCheckPath is the public home page, while /api/health performs a Gemini model call when configured. This avoids vendor spend from platform liveness checks, which is good. The public /api/health route remains callable by anyone and should be changed to a cheap internal readiness check.

9. Shared data model

lib/types.ts defines ChatMessage, ChatRecord, AdminUser, Session, PriceEntry, KnowledgeEntry, FeedbackEntry, Subscriber, ContactEntry, AppSettings, Analytics and Database. This central schema is the contract connecting UI tabs, APIs and persistence. It is compile-time only. Runtime payload validation is handwritten and uneven; there is no Zod, JSON Schema or generated OpenAPI document.

10. Dependency and module boundaries

- * lib/env.ts is the only intended provider-secret resolution layer.
- * lib/ai.ts may read settings and knowledge through lib/db.ts, so the intelligence layer is state-aware.
- * app/api/chat/route.ts is the orchestration center and directly coordinates DB, search, Gemini, Cloudflare and language modules.
- * lib/db.ts imports lib/pg-store.ts for mirror scheduling, while instrumentation imports both for hydration.
- * Client components import only browser-safe types, language constants and site context. Server provider modules are not imported by client components.

11. Construction conclusion

The framework succeeds through consolidation. One typed Next.js project owns the screen, HTTP edge, AI adapters and persistence. This reduces deployment complexity and makes the call paths easy to inspect. It also concentrates security, state consistency and scaling risk into one Node process. The design is appropriate for a focused prototype and must evolve before multi-instance national traffic.

Authorship and control statement

This volume was prepared, authored and issued solely under the name of Thaddeus Nii Teiko Tagoe. This architectural reconstruction records how the application is assembled from source and which boundaries hold it together. The repository records identify Thaddeus Tagoe as the founder, sole designer, developer and maintainer of AgriAI.